BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Nguyễn Đăng Khoa
MSSV / Lớp	3123410168 / DCT1232

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

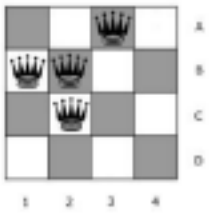
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```

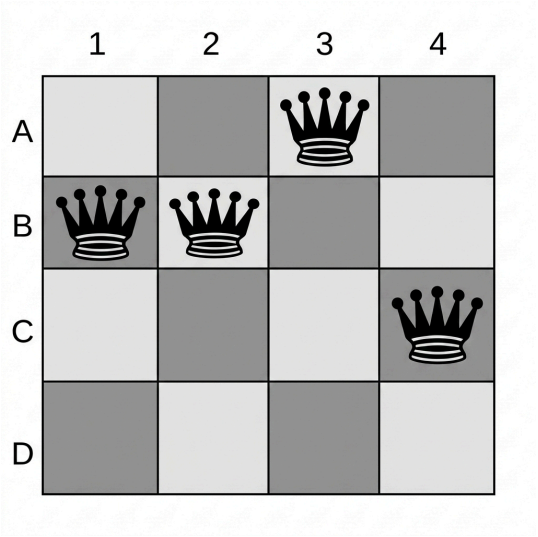


Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Trạng thái khởi đầu có 4 quân hậu tại các ô: (B, 1), (B, 2), (C, 2), (A, 3) với số cặp xung đột là 4. Một vòng lặp chọn nước đi hợp lệ của cờ vua giúp giảm số xung đột nhiều nhất (tối ưu nhất) là di chuyển quân hậu tại ô (C, 2) sang cột 4 ở ô (C, 4).

-> Vị trí của từng quân hậu sau 1 vòng lặp: (B, 1), (B, 2), (A, 3), (C, 4).



b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☑ B) Cực trị cục bộ (local optimum)

☐ C) Cực trị toàn cục (global optimum)

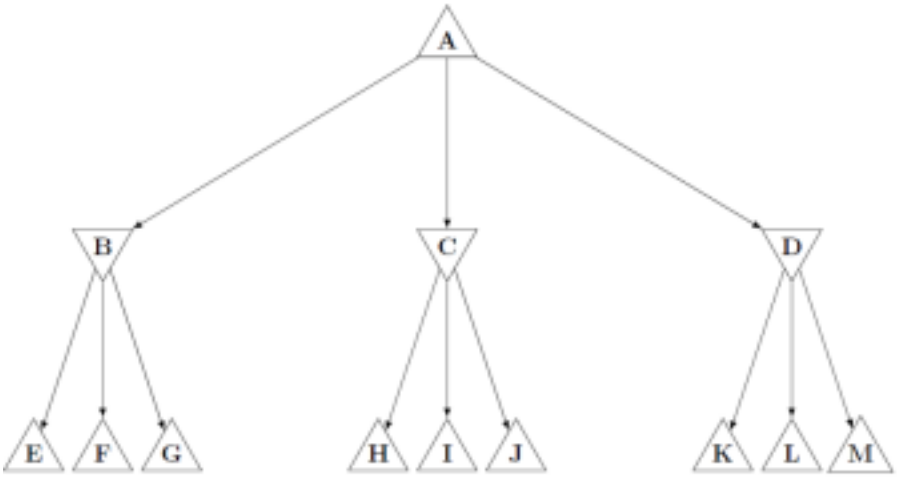
☑ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị

hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Xét cây con D trước (Min node): các nút con K(23), L(20), M(26) => D trả về min(23, 20, 26) = 20. Nút gốc A (Max node) cập nhật alpha = 20.

Xét tiếp cây con C (Min node): nút con đầu tiên H(17). Do C là Min node nên giá trị của C <= 17. Vì alpha = 20 >= 17, nhánh C không thể trả về giá trị lớn hơn 20 cho nút MAX A. Do đó, các nút còn lại của C là I và J BỊ CẮT TỈA (không được thăm).

Xét tiếp cây con B (Min node): nút con đầu tiên E(5). Giá trị của B <= 5. Do alpha = 20 >= 5, nhánh B không thể trả về giá trị lớn hơn 20 cho A. Do đó, các nút con còn lại của B là F và G BỊ CẮT TỈA (không được thăm).

Danh sách các nút KHÔNG được thăm do bị cắt tỉa: I, J, F, G.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative

deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

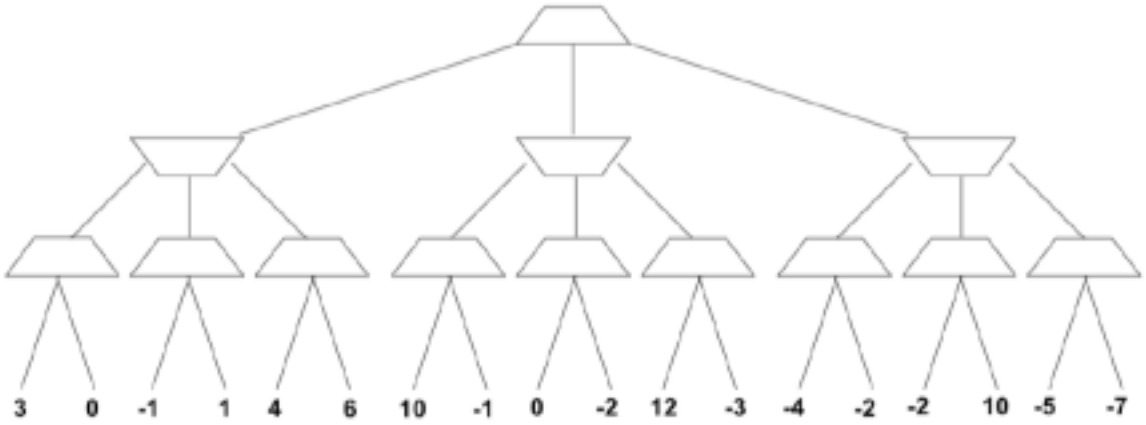
☐ Có

☑ Không

Giải thích: Hàm đánh giá f(s) ở độ sâu nông chỉ là ước lượng (heuristic) và có thể không phản ánh đúng giá trị thực tế của cây con ở các tầng sâu hơn. Sắp xếp nhánh dựa trên hàm đánh giá nông chỉ giúp tối ưu hóa thứ tự duyệt để tăng số lượng cắt tỉa, nhưng nếu dừng sớm hoặc cắt tỉa dựa trên giá trị ước lượng sai thì có thể bỏ qua nước đi tối ưu thực sự của Minimax đầy đủ.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

1. Cây có 4 tầng: Tầng 0 (MAX - Gốc), Tầng 1 (MIN), Tầng 2 (MAX), Tầng 3 (Lá).

- Nhánh trái nhất: các lá 3, 0 -> MAX chọn 3. Các lá -1, 1 -> MAX chọn 1. Các lá 4, 6 -> MAX chọn 6.

  - MIN ở nhánh 1 tầng 1 chọn min(3, 1, 6) = 1.

  - Gốc MAX cập nhật alpha = 1.

- Nhánh giữa:

  - Nút MAX đầu tiên: lá 10, -1 -> trả về 10.

  - Nút MIN ở tầng 1 cập nhật beta = 10.

  - Nút MAX thứ hai: lá 0, -2 -> trả về 0. Nút MIN ở tầng 1 cập nhật beta = min(10, 0) = 0.

- Vì beta = 0 <= alpha = 1, thực hiện CẮT TỈA (Pruning) ở nhánh thứ 3 của nút MIN này (lá 12, -3 không cần xét).

- Nút MIN thứ 2 trả về 0.

- Nhánh phải:

- Nút MAX đầu tiên: lá -4, -2 -> trả về -2.

- Nút MIN ở tầng 1 cập nhật beta = -2.

- Vì beta = -2 <= alpha = 1, CẮT TỈA toàn bộ các nhánh còn lại của nút MIN thứ 3 (không xét các lá -2, 10 và -5, -7).

- Nút MIN thứ 3 trả về -2.

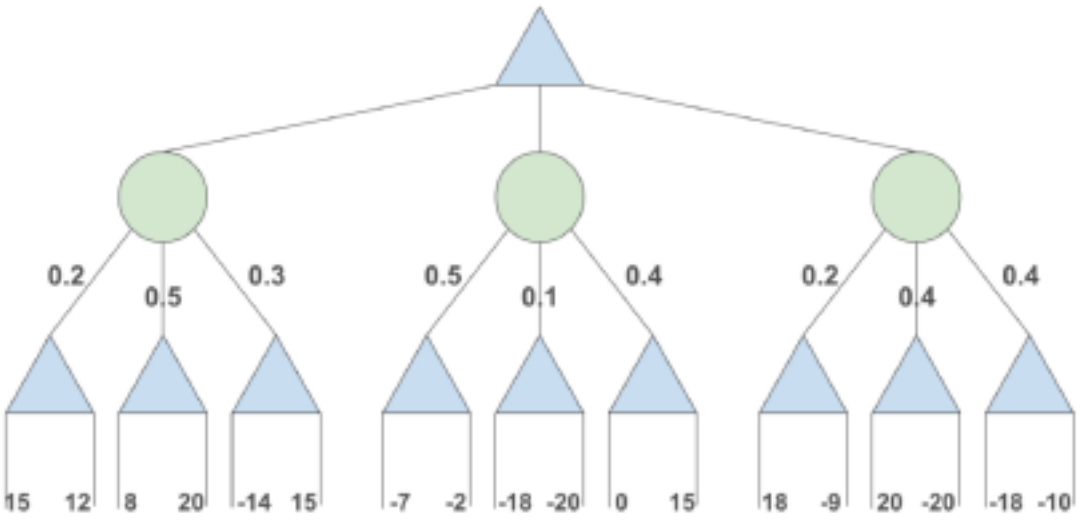
Kết quả:

- Giá trị Minimax tại nút gốc MAX = 1.

- Nước đi tối ưu cho MAX: Chọn nhánh bên trái (trả về giá trị 1).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Tính giá trị tại các nút:

- Nút MAX tầng 2 (từ trái sang phải):

+ Cụm 1: max(15, 12) = 15; max(8, 20) = 20; max(-14, 15) = 15.

+ Cụm 2: max(-7, -2) = -2; max(-18, -20) = -18; max(0, 15) = 15.

+ Cụm 3: max(18, -9) = 18; max(20, -20) = 20; max(-18, -10) = -10.

- Các Chance node tầng 1 (tính kỳ vọng):

+ Chance 1 = 0.2*15 + 0.5*20 + 0.3*15 = 3 + 10 + 4.5 = 17.5.

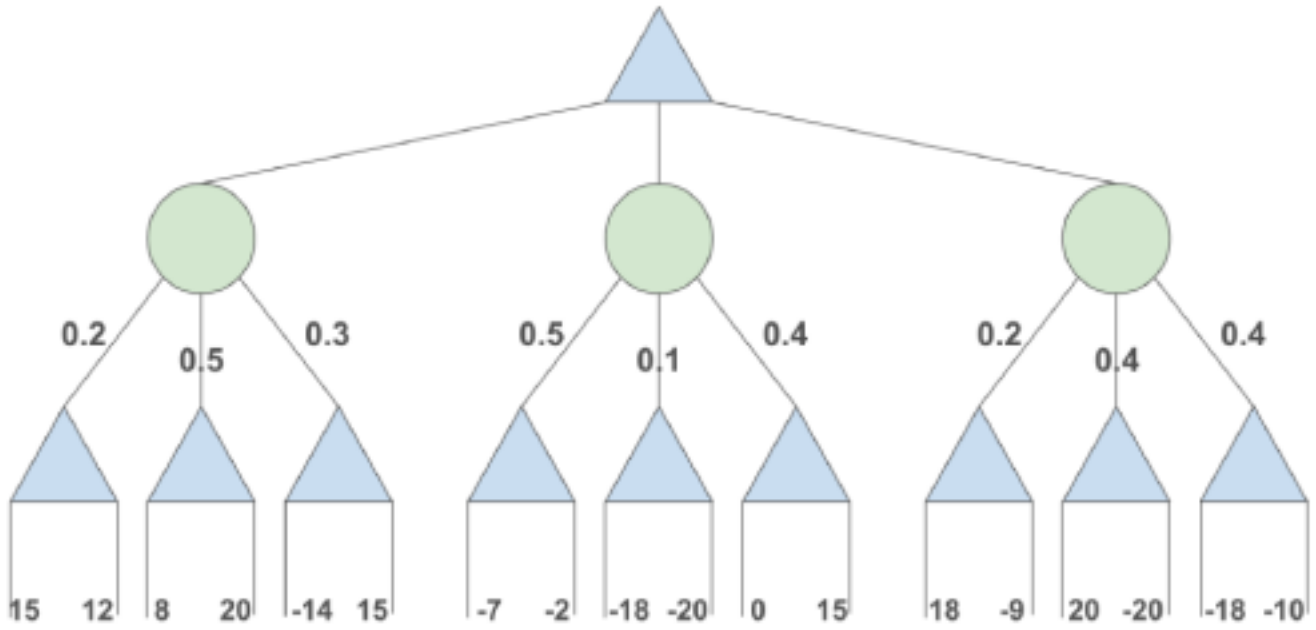
+ Chance 2 = 0.5*(-2) + 0.1*(-18) + 0.4*15 = -1 - 1.8 + 6 = 3.2.

+ Chance 3 = 0.2*18 + 0.4*20 + 0.4*(-10) = 3.6 + 8 - 4 = 7.6.

- Nút Gốc MAX = max(17.5, 3.2, 7.6) = 17.5.

b) Nếu chưa biết thêm giới hạn nào về giá trị lá: Không thể cắt tỉa bất kỳ nút nào trong cây Expectimax. Vì các giá trị lá có thể là vô cùng lớn (unbounded), một lá chưa khám phá ở nhánh sau hoàn toàn có thể kéo giá trị kỳ vọng của chance node đó lên rất cao, làm thay đổi quyết định ở nút MAX gốc.

Phương pháp này KHÔNG luôn đảm bảo trả về cùng kết quả chính xác như Expectimax/Minimax đầy đủ nếu chưa biết giới hạn của các nút lá. Vì không có giới hạn trên/dưới cho giá trị lá, một lá chưa khám phá ở nhánh sau hoàn toàn có thể có giá trị cực kỳ lớn hoặc cực kỳ nhỏ, làm thay đổi hoàn toàn giá trị kỳ vọng của chance node và dẫn đến quyết định khác ở nút gốc.
vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Giới hạn giá trị lá: V_min = -20, V_max = 20.

- Chance 1 = 17.5 (đã tính đầy đủ). Nút MAX gốc có giá trị tạm thời = 17.5.

- Xét Chance 2:

+ Nhánh 1: MAX = -2 (xác suất 0.5).

+ Nhánh 2: MAX = -18 (xác suất 0.1).

+ Giá trị tối đa mà Chance 2 có thể đạt được (ngay cả khi nhánh 3 đạt tối đa V_max = 20) là: 0.5*(-2) + 0.1*(-18) + 0.4*20 = -1 - 1.8 + 8 = 5.2.

+ Vì 5.2 < 17.5, Chance 2 KHÔNG THỂ vượt qua 17.5. Do đó ta CẮT TỈA nhánh 3 của Chance 2 (các lá 0, 15).

- Xét Chance 3:

+ Nhánh 1: MAX = 18 (xác suất 0.2).

+ Nhánh 2: MAX = 20 (xác suất 0.4).

+ Giả sử nhánh 3 đạt tối đa V_max = 20, giá trị tối đa Chance 3 = 0.2*18 + 0.4*20 + 0.4*20 = 3.6 + 8 + 8 = 19.6 > 17.5, nên phải xét tiếp nhánh 3.

+ Xét lá -18 của nhánh 3, lá tiếp theo là -10 -> MAX = -10.

+ Chance 3 thực tế = 7.6.

Kết quả: Cắt tỉa được nhánh 3 của Chance node 2. Nút gốc vẫn chọn 17.5.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

So sánh Forward Checking (FC) và AC-3:

- Phạm vi lan truyền: FC chỉ kiểm tra tính nhất quán giữa biến vừa được gán với các biến chưa được gán có liên kết trực tiếp (1 bước). AC-3 duy trì tính nhất quán cung (Arc Consistency) trên toàn bộ đồ thị ràng buộc, lan truyền sự thay đổi qua nhiều bước cho đến khi không còn cung nào bị vi phạm.

- Thời điểm thực hiện: FC chạy sau mỗi bước gán giá trị cho một biến. AC-3 có thể chạy ở bước tiền xử lý (trước khi tìm kiếm) hoặc sau mỗi bước gán (MAC - Maintaining Arc Consistency).

- Chi phí tính toán: FC có chi phí tính toán thấp hơn ở mỗi bước. AC-3 có chi phí cao hơn đáng kể cho mỗi bước nhưng giúp phát hiện sớm xung đột và thu hẹp không gian tìm kiếm nhiều hơn.

b) Lý do sử dụng các heuristic trong CSP:

- Minimum Remaining Values (MRV)

MRV chọn biến có ít giá trị hợp lệ còn lại nhất ("Fail-first principle"). Việc chọn biến khó nhất giúp phát hiện điểm cụt (dead-end) sớm nhất có thể, tránh đi sâu vào các nhánh tìm kiếm vô nghiệm.

- Least Constraining Value (LCV)

LCV chọn giá trị ít loại bỏ giá trị của các biến lân cận nhất ("Fail-last principle"). Việc này tối đa hóa khả năng tìm thấy nghiệm ngay ở lần thử đầu tiên mà không phải quay lùi.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

a) Mô hình hóa CSP:

- Biến: A, B, C, D, E. Mỗi biến biểu diễn cặp (Đường băng, Khung giờ).

- Miền giá trị ban đầu: D = {QT1, QT2, QT3, QT4, NĐ1, NĐ2, NĐ3, NĐ4} (QT: Quốc tế, NĐ: Nội địa).

- Ràng buộc đơn (Unary):

+ B: Khung giờ 1 => B ∈ {QT1, NĐ1}.

+ D: Khung giờ >= 3 => D ∈ {QT3, QT4, NĐ3, NĐ4}.

+ A: Khung giờ <= 2 => A ∈ {QT1, QT2, NĐ1, NĐ2}.

- Ràng buộc nhị phân (Binary):

+ D hạ cánh trước C cất cánh: Time(D) < Time(C).

+ Không trùng khung giờ và đường băng: Với mọi X ≠ Y, nếu Runway(X) == Runway(Y) thì Time(X) ≠ Time(Y). **Bổ sung hai ràng buộc sau:**

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

b) Miền mới:

- D_A = {1, 2}, D_B = {1, 2, 3, 4}, D_C = {1, 2, 3, 4} (trên đường băng Quốc tế).

- D_D = {1, 2, 3, 4}, D_E = {1, 2, 3, 4} (trên đường băng Nội địa).

Đồ thị ràng buộc:

- Nhóm Quốc tế: Cạnh giữa (A, B), (B, C), (A, C) với ràng buộc khác nhau (≠).

- Nhóm Nội địa: Cạnh giữa (D, E) với ràng buộc khác nhau (≠).

- Liên nhóm: Cạnh giữa D và C với ràng buộc D < C.

c) Áp dụng ràng buộc Unary & Arc Consistency:

- Ràng buộc Unary:

+ B phải ở giờ 1 => D_B = {1}.

+ A phải <= 2 => D_A = {1, 2}.

+ D phải >= 3 => D_D = {3, 4}.

+ D_C = {1, 2, 3, 4}, D_E = {1, 2, 3, 4}.

- Áp dụng Arc Consistency (AC-3):

+ Do D_B = {1} và A, C dùng chung đường băng QT:

* Cung (A, B): B=1 loại 1 khỏi D_A => D_A = {2}.

* Cung (C, B): B=1 loại 1 khỏi D_C => D_C = {2, 3, 4}.

+ Cung (C, A): A=2 loại 2 khỏi D_C => D_C = {3, 4}.

+ Cung (C, D) với D < C: D ∈ {3, 4} nên C phải > D. Nếu D=3 thì C có thể là 4. Nếu D=4 thì C không có giá trị > 4. Do đó D=4 bị loại khỏi D_D => D_D = {3}.

+ Cung (C, D) lại yêu cầu C > 3 => D_C = {4}.

+ Cung (E, D) trên NĐ: D=3 loại 3 khỏi D_E => D_E = {1, 2, 4}.

Miền còn lại sau AC-3: D_A = {2}, D_B = {1}, D_C = {4}, D_D = {3}, D_E = {1, 2, 4}.
unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

d) Forward Checking + Backtracking Search:

- Miền sau khi áp dụng Unary: D_A = {1, 2}, D_B = {1}, D_C = {1, 2, 3, 4}, D_D = {3, 4}, D_E = {1, 2, 3, 4}.

1. Chọn B (MRV = 1): Gán B = 1.

- FC: Loại 1 khỏi A => D_A = {2}. Loại 1 khỏi C => D_C = {2, 3, 4}.

2. Chọn A (MRV = 1, D_A = {2}): Gán A = 2.

- FC: Loại 2 khỏi C => D_C = {3, 4}.

3. Chọn D (MRV = 2, D_D = {3, 4}): Gán D = 3.

- FC: Loại 3 khỏi E => D_E = {1, 2, 4}. Cần D < C, giữ D_C = {4}.

4. Chọn C (MRV = 1, D_C = {4}): Gán C = 4.

- FC: Không làm giảm thêm miền nào.

5. Chọn E (MRV = 3, D_E = {1, 2, 4}): Gán E = 1.

Thứ tự các phép gán thành công (không bị backtrack): (B, 1) -> (A, 2) -> (D, 3) -> (C, 4) -> (E, 1).

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

Biến	**Giờ 1**	**Giờ 2**	**Giờ 3**	**Giờ 4**

A	~~1~~	**2 (Gán)**	~~3~~	~~4~~
B	**1 (Gán)**	~~2~~	~~3~~	~~4~~
C	~~1~~	~~2~~	~~3~~	**4 (Gán)**
D	~~1~~	~~2~~	**3 (Gán)**	~~4~~
E	**1 (Gán)**	~~2~~	~~3~~	~~4~~

Quá trình gán giá trị theo Forward Checking + Backtracking Search:

- Miền ban đầu sau khi áp dụng các ràng buộc Unary:

+ D_A = {1, 2}, D_B = {1}, D_C = {1, 2, 3, 4}, D_D = {3, 4}, D_E = {1, 2, 3, 4}.

1. Chọn biến B (MRV = 1): Gán (B, 1).

- FC (bật từ B = 1): Loại 1 khỏi A -> D_A = {2}. Loại 1 khỏi C -> D_C = {2, 3, 4}.

2. Chọn biến A (MRV = 1, D_A = {2}): Gán (A, 2).

- FC (bật từ A = 2): Loại 2 khỏi C -> D_C = {3, 4}. Các biến khác không bị ảnh hưởng trực tiếp.

3. Chọn biến C hoặc D (cùng MRV = 2: D_C = {3, 4}, D_D = {3, 4}). Quy tắc hòa chọn ký tự nhỏ hơn -> Chọn C, gán giá trị nhỏ hơn trước -> Gán (C, 3).

- FC (bật từ C = 3): Ràng buộc D < C yêu cầu D phải < 3. Tuy nhiên, D_D = {3, 4}, do đó loại cả 3 và 4 khỏi D_D -> D_D = {} (rỗng).

- Phát hiện DEAD END ở biến D!

4. BACKTRACK: Hoàn tác phép gán (C, 3). Khôi phục D_C = {3, 4}, D_D = {3, 4}.

5. Thử giá trị tiếp theo cho C -> Gán (C, 4).

- FC (bật từ C = 4): Ràng buộc D < C yêu cầu D phải < 4. Loại 4 khỏi D_D -> D_D = {3}. D_E giữ nguyên = {1, 2, 3, 4}.

6. Chọn biến D (MRV = 1, D_D = {3}): Gán (D, 3).

- FC (bật từ D = 3): D và E cùng dùng đường băng nội địa nên E ≠ D. Loại 3 khỏi E -> D_E = {1, 2, 4}.

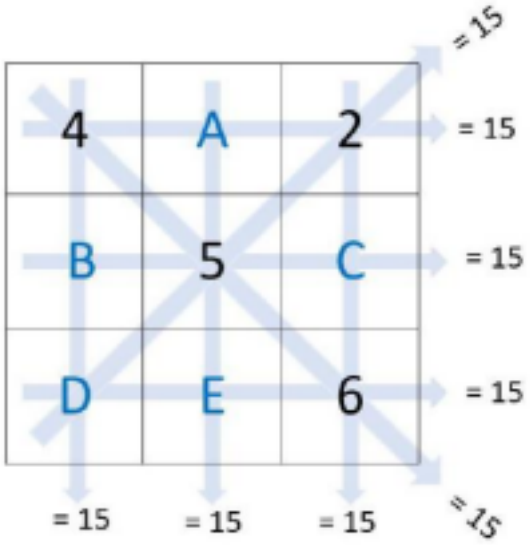
7. Chọn biến E (MRV = 3, D_E = {1, 2, 4}), chọn giá trị nhỏ nhất trước -> Gán (E, 1).

Thứ tự các cặp (biến, giá trị) phát sinh thực tế: (B, 1) -> (A, 2) -> (C, 3) [Dead End] -> Hoàn tác (C, 3) -> (C, 4) -> (D, 3) -> (E, 1).

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô

đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

a) Mô hình hóa CSP:

- Biến: A, B, C, D, E.

- Miền giá trị: D_A = D_B = D_C = D_D = D_E = {1, 2, 3, 4, 5, 6, 7, 8, 9} \ {2, 4, 5, 6} = {1, 3, 7, 8, 9}.

- Ràng buộc:

  + alldiff(A, B, C, D, E, 2, 4, 5, 6).

  + Hàng 1: 4 + A + 2 = 15 => A = 9.

  + Cột 1: 4 + B + D = 15 => B + D = 11.

  + Hàng 2: B + 5 + C = 15 => B + C = 10.

  + Hàng 3: D + E + 6 = 15 => D + E = 9.

  + Cột 2: A + 5 + E = 15 => A + E = 10.

  + Cột 3: 2 + C + 6 = 15 => C = 7.

  + Đường chéo chính: 4 + 5 + 6 = 15 (thỏa mãn).

  + Đường chéo phụ: 2 + 5 + D = 15 => D = 8.

b) Đồ thị ràng buộc nhị phân (bỏ alldiff):

- Đỉnh: A, B, C, D, E.

- Cạnh ràng buộc:

+ (A, E) với A + E = 10.

+ (B, D) với B + D = 11.

+ (B, C) với B + C = 10.

+ (D, E) với D + E = 9.

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

c) Thu hẹp miền bằng AC-3 và tìm nghiệm:

- Từ các ràng buộc đơn trực tiếp:

+ C = 7, D = 8, A = 9.

- Áp dụng vào các biến còn lại:

+ B + C = 10 => B + 7 = 10 => B = 3.

+ D + E = 9 => 8 + E = 9 => E = 1.

- Kiểm tra các ràng buộc khác:

+ B + D = 3 + 8 = 11 (Thỏa mãn).

+ A + E = 9 + 1 = 10 (Thỏa mãn).

Nghiệm duy nhất của Magic Square: A = 9, B = 3, C = 7, D = 8, E = 1.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

a) Dùng MRV + LCV:

- Miền ban đầu: AJ = {Đỏ, Vàng, Xanh lá} (3), Sunny = {Đỏ, Vàng, Hồng} (3), Ellyse = {Đỏ, Hồng} (2).

- Bước 1: Chọn biến theo MRV => Chọn Ellyse (kích thước miền = 2).

+ Xét giá trị cho Ellyse bằng LCV:

* Nếu chọn Đỏ: loại Đỏ khỏi AJ và Sunny (loại 2 lựa chọn).

* Nếu chọn Hồng: loại Hồng khỏi Sunny (loại 1 lựa chọn).

* LCV chọn Hồng trước (ít gây ràng buộc hơn).

+ Gán Ellyse = Hồng.

+ Cập nhật miền: Sunny = {Đỏ, Vàng}, AJ = {Đỏ, Vàng, Xanh lá}.

- Bước 2: Chọn biến theo MRV => Chọn Sunny (kích thước miền = 2).

+ Xét giá trị cho Sunny bằng LCV:

* Chọn Vàng: loại Vàng khỏi AJ (loại 1 lựa chọn).

* Chọn Đỏ: loại Đỏ khỏi AJ (loại 1 lựa chọn).

* Hòa LCV: Phá hòa theo thứ tự cầu vồng (Đỏ < Vàng) => Chọn Đỏ.

+ Gán Sunny = Đỏ.

+ Cập nhật miền: AJ = {Vàng, Xanh lá}.

- Bước 3: Chọn AJ, chọn Vàng (hoặc Xanh lá). Gán AJ = Vàng.

Số lần backtrack: 0 lần. Thứ tự gán: Ellyse = Hồng -> Sunny = Đỏ -> AJ = Vàng.

b) Nếu thay MRV bằng Maximum Remaining Values (chọn biến có nhiều giá trị nhất) và LCV bằng Most Constraining Value (MCV - chọn giá trị gây nhiều ràng buộc nhất), hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

b) Thay bằng Maximum Remaining Values + Most Constraining Value (MCV):

- Bước 1: Chọn biến có miền lớn nhất (AJ hoặc Sunny, hòa chọn AJ). Gán giá trị gây ràng buộc nhiều nhất (Đỏ).

+ Gán AJ = Đỏ. Miền còn lại: Sunny = {Vàng, Hồng}, Ellyse = {Hồng}.

- Bước 2: Chọn Sunny (miền = 2) hoặc Ellyse (miền = 1). Theo Maximum Remaining Values chọn Sunny (miền lớn hơn).

+ MCV chọn Hồng (gây ràng buộc loại Hồng của Ellyse).

+ Gán Sunny = Hồng.

+ Cập nhật miền: Ellyse = {} (rỗng) -> DEAD END!

- Phải BACKTRACK! Hoàn tác Sunny = Hồng, đổi sang Sunny = Vàng.

+ Gán Ellyse = Hồng.

Số lần backtrack: 1 lần.

Giải thích: Heuristic Maximum Remaining Values và MCV đi ngược lại với nguyên tắc thiết kế tối ưu của CSP. Chọn biến có nhiều giá trị nhất sẽ làm hoãn việc giải quyết các biến nguy hiểm (dễ cụt) và chọn giá trị gây ràng buộc nhất sẽ làm kiệt huệ miền giá trị của các biến khác, dẫn đến bế tắc và buộc phải backtrack.